Department of Computer Science

BSCCS Final Year Project Report

2025-2026

25CS122

GitReader: An Intelligent Repository Comprehension System Through Contextual Dialogue

(Volume __ of __)

Student Name : **FAN Tianrui**

Student No. : **57853947**

Programme Code : **BSCCCU4**

Supervisor : **Prof MA, Chen**

1st Reader : **Prof LU, Zhichao**

2nd Reader : **Prof LIANG, Weifa**

For Official Use Only

Student Final Year Project Declaration

I have read the project guidelines and I understand the meaning of academic dishonesty, in particular plagiarism and collusion. I hereby declare that the work I submitted for my final year project, entitled:

GitReader: An Intelligent Repository Comprehension System Through Contextual Dialogue

does not involve academic dishonesty. I give permission for my final year project work to be electronically scanned and if found to involve academic dishonesty, I am aware of the consequences as stated in the Project Guidelines.

Student Name:	FAN Tianrui	Signature:	FAN Tianrui
Student ID:	57853947	Date:	March 29, 2026

Table of Contents

1. Extended Abstract ---------- 5

2. Acknowledgments ---------- 6

3. Introduction ---------- 7

3.1. Background 7

3.2. Problem Definition 7

3.3. Scope 8

3.4. Significance & Motivation 9

3.5. Objectives and Expected Deliverables 9

3.6. Creative Aspects of the Project 10

4. Literature Review ---------- 12

4.1. Documentation as Code and Developer Portals 12

4.2. AI-assisted Code Understanding and Repository Q&A (RAG) 13

4.3. Revolutionary Impact 13

5. Proposed design and system ---------- 14

5.1. System Overview 14

5.2. System Components 14

5.3. User Interaction Flow 15

5.4. Pipeline Refinement 16

6. Detailed Methodology and Implementation ---------- 18

6.1. Repository Ingestion and File Filtering 18

6.2. Repository Structure Extraction and RepoMap Construction 18

6.3. Code Parsing and Semantic Chunking 19

6.4. CodeGraphBuilder (Repository Structural Graph) 19

6.5. Community Detection and Module Summarization 20

6.6. Embedding and Hybrid Index Construction 21

6.7. Community-first Retrieval and Evidence Selection 21

6.8. Backend API and Task Orchestration 22

6.9. Storage and Deployment Integration 22

7. Testing and Evaluation ----------23

7.1. Testing Environment and Dataset 23

7.2. Quantitative Performance Analysis 25

8. Conclusion and Critical Review ----------27

8.1. Summary of Achievements 27

8.2. Critical Review 27

8.3. Learning Experiences 29

8.4. Future Work 30

9. Appendices ----------31

9.1. Appendices A Monthly Logs 31

9.2. Appendices B Database Schema and Cloud Storage 32

9.3. Appendices C Backend API List 33

10. References ----------34

1. Extended Abstract

GitReader aims to address the practical difficulty of understanding and reusing large public repositories whose documentation is incomplete, outdated, or too shallow for onboarding, maintenance, and adaptation. Direct repository summarisation with LLMs is often unreliable for large projects because important structural context is often missed, and unsupported explanations can be introduced. This project therefore investigates a structure-first, evidence-grounded approach that generates repository documentation and supports repository-specific question answering using source-derived context and other external resources.

The final system is a deployable web application that combines automated wiki generation with agent-based repository dialogue. After cloning a repository snapshot, GitReader filters irrelevant files, parses source code with Tree-sitter and RepoMapper, constructs a compact repository map and a lightweight code graph, and applies Leiden community detection to identify coherent functional modules. Based on this structural context, the system first produces a hierarchical table of contents linked to real source files and then expands each section into Markdown text and Mermaid diagrams. For interactive exploration, GitReader provides an agent that can interpret repository-related questions, break them into repository-understanding steps, invoke repository-aware tools such as search and retrieval over code and text chunks, and synthesize evidence-grounded responses with explicit source references.

This project implements a complete web system that supports project uploads through a single interface, automatic wiki generation, evidence-based browsing, and an intelligent agent to reply to users' questions. The entire workflow has been validated on multiple GitHub repos and demonstrated that the structure-first approach improves documentation reliability. It's particularly useful for concrete problems involving architectural understanding and excavating connections across files. The current system still has some limitations, such as using a heuristic rather than a compiler-level approach for dependency resolution, a lack of commit-level incremental updates, and a lack of large-scale quantitative evaluation.

2. Acknowledgments

First and foremost, I would like to express my sincere gratitude to my supervisor, **Prof. MA Chen**, for his invaluable guidance, patience, and continuous support throughout this project.

3. Introduction

3.1. Background

Large GitHub projects are becoming more and more important in the modern IT ecosystem. However, users often struggle to understand and effectively reuse these projects -- Documentation is frequently outdated, incomplete, or unable to address the special problems, making it hard for new contributors to get started or reuse them. For the non-technical personnel, undocumented issues can be especially disruptive.

Purely LLM-based Q&A methods are generally unreliable, as LLMs have limited context windows and may also generate illusory information without sufficient supporting evidence. Therefore, the project adopted a workflow that combines the repository itself with external evidence to ensure the content's authenticity to the greatest extent possible.

GitReader: An Intelligent Repository Comprehension System Through Contextual Dialogue aims to address this gap by building an AI-powered web application with two main functions: generating detailed documentation to help users understand the project’s general outline and detailed modules, functions, and dependencies; and providing an interactive AI agent that can answer repository-specific questions accurately based on source code and other relevant online references. The overall goal is to reduce the cost of project start and reuse, improve development efficiency, and enhance user experience.

3.2. Problem Definition

The central problem is the difficulty of understanding and reusing large GitHub repositories. The existing project documentation is often overly simplified, outdated, or missing entirely, serving only IT professionals. Users who attempt to reuse or contribute to these projects find it difficult to start or resolve issues they encounter while using them independently, particularly when facing undocumented features or systematic problems. This problem influences developers and non-technical users (such as researchers or engineers from different fields) who want to reuse and

expand those tools to their respective fields. Even for technical people, this is also a troublesome matter.

3.3. Scope

To bridge the gap between developers' and users' focus, a system that combines repository-structure-based understanding with AI is required. Such a system will analyze and understand repository structure and specific code, enabling users to ask domain-relevant questions and receive accurate, repository-based answers.

In-Scope

- Public GitHub Repository Ingestion: Automated cloning and preprocessing of public repositories based on the HEAD state
- Structural Code Analysis: Multi-language AST parsing (Python, TS, Go, etc.) at the function and class level using Tree-sitter, including heuristic CodeGraph construction
- Automated Wiki Documentation System:
  - Structure Planning: Logic-driven Table of Contents (ToC) generation using Leiden community detection for functional module discovery
  - Content Generation: Parallel generation of Markdown pages and Mermaid diagrams with explicit repository evidence grounding
- Advanced Hybrid RAG Pipeline: Repository-grounded Q&A featuring HyDE query expansion, Hybrid Search (Dense FAISS + Sparse BM25), and MMR-based diversity re-ranking
- Scalable System Orchestration: Asynchronous task management with task-level isolation, concurrent processing, and integrated cloud storage

Out-of-Scope

- **Exhaustive "Line-by-Line" Annotation**: To avoid LLM hallucinations and excessive token costs, the system does not provide an exhaustive line-by-line explanation of the entire codebase. It focuses on providing evidence-grounded explanations for specific code spans retrieved during Q&A or Wiki generation

- **Dynamic Analysis and Runtime Profiling**: The comprehension is strictly based on static analysis (AST and CodeGraph); it does not include dynamic execution, runtime profiling, or live debugging of the code
- **Incremental Synchronization**: Automatic, real-time documentation updates based on Git commit diffs are excluded. Currently, the system processes a repository snapshot at its specific HEAD state for each task

3.4. Significance & Motivation

GitHub is the center of the modern open-source community, which houses tens of thousands of valuable projects. However, developers and users differ in their focus: one focuses on the project itself, while the other focuses more on how to use it. Specifically, one focuses on the core code, while the other focuses on practical applications. As a consequence, many excellent projects lack detailed documentation and get less attention.

Furthermore, developers cannot anticipate and resolve all exceptions, and users' environments vary. So, users often encounter some bizarre errors that are not mentioned in the document. While technical personnel can quickly resolve most of them, deeper code and structural problems often require a thorough understanding of the project itself and targeted modifications, which is extremely time-consuming and inefficient. For non-technical personnel, this approach is completely impossible.

3.5. Objectives and Expected Deliverables

3.5.1. Objectives

1. **Repository Understanding**
   The public GitHub repository input by the user is cloned and preprocessed to construct a structured representation that can be used for downstream inference, including a RepoMap, code parsing results, dependency/call relationship graphs, and feature community division results.
2. **Auto Wiki Documentation Generation**
   It automatically generates a hierarchical wiki table of contents (ToC) and generates browsable page content (Markdown) for each chapter, while also

generating a Mermaid diagram to represent module relationships and data flow.

3. **Repository-grounded Q&A**
   Build a local index (FAISS + BM25) to enable hybrid retrieval and reordering, ensuring that question-and-answer responses are based on evidence within the repository.

3.5.2. Expected Deliverables

1. **Web frontend**
   - **Repo Management Interface**: Supports GitHub URL input, task triggering, and real-time progress tracking
   - **Wiki Browser:** Multiple-level navigation for the generated documentation
   - **Contextual Chat Interface:** An interactive Q&A and Agent UI featuring "Evidence Cards" and source citations for the user to check
2. **Backend API**
   - **Analysis & Generation Pipeline:** Cloning the whole project, Tree-sitter parsing, and concurrent Wiki generation
   - **Intelligence Engine:** Implements Leiden-based community detection for module discovery and structural planning
   - **Advanced RAG API:** A robust retrieval pipeline that integrates HyDE query expansion, Hybrid Search, and MMR re-ranking

3.6. Creative Aspects of the Project

The innovation of the project lies in shifting the method of repository comprehension from "unconstrained LLM summarization" to a "structured, evidence-driven generation and retrieval" system. Unlike the traditional general-purpose AI assistants, GitReader ensures technical grounding through the following creative approaches:

1. **Leiden-Enhanced Structure-First Wiki Planning**
   Instead of letting LLM blindly guess the repository's structure, the system employs Tree-sitter parsing and a heuristic code graph to model internal dependencies. By applying the Leiden community detection algorithm to this

graph, the system can identify semantically coherent modules. These modules guide the LLM in generating a Table of Contents (ToC) constrained by a `valid_file_list`, effectively eliminating the "path hallucinations" problem

2. **Hybrid RAG with Diverse Evidence Retrieval**
   To maximize accuracy and recall in repository-specific Q&A, the system implements a sophisticated Hybrid RAG pipeline. It fuses dense vector search (FAISS) with sparse keyword search (BM25) using a weighted scoring mechanism (Douze et al., 2024). Furthermore, it utilizes HyDE (Hypothetical Document Embedding) for query expansion (Gao et al., 2023) and MMR (Maximum Marginal Relevance) to ensure the retrieved evidence set is both relevant and diverse, significantly reducing the risk of "AI hallucinations".
3. **Scalable Engineering Closure & Task-Level Isolation**
   The generated content (wiki structure/pages) is stored in Cloudflare R2 in JSON format, conversation and task metadata are entered into Supabase, and the local index is persisted in a Fly.io volume, thus realizing a complete system loop and ensuring the system can run stably and persistently.

4. Literature Review

4.1. Documentation as Code and Developer Portals

Modern documentation tools follow the "documentation as code" philosophy (O'Dell, 2023). They treat documentation as the most vital guideline in the development lifecycle, versioning it alongside the code and integrating it into the CI pipeline. This methodology greatly improves the relevance of documentation to the codebase and its long-term maintainability.

- **Swimm:** Swimm is the exemplification of the concept of continuous documentation (Swimm, n.d.). Linking documentation sections to code blocks addresses inconsistencies between code and documentation. When relevant code changes occur, the Swimm plugin will automatically detect the updates and remind developers to update the documentation. While Swimm reduces the maintenance delay, the system itself still requires developers to build, serving only as a reminder and control system.
- **Backstage:** Backstage is an open-sourced framework that provides a platform for developers to build portals (Spotify Engineering, 2020). Through a central software catalog, it allows large organizations to manage all documentation in one place. Backstage provides a global view of the project architecture and the relationships among all components for developers.

Shared Limitations

While these two tools all tightly integrate documentation maintenance with the development process, they share a common limitation: human maintenance. While they provide pre-written templates, developers still must spend time and effort on manual maintenance. GitReader commits to reducing the burden of documentation for developers so that they can fully concentrate on code development.

4.2. AI-assisted Code Understanding and Repository Q&A (RAG)

With the rapid development of **LLM**, the new AI tools are moving from simple auto code completion to a deeper **understanding of the entire codebase**. At the heart of this shift lies **Retrieval-Augmented Generation** (Lewis et al., 2020).

- **Sourcegraph Cody:** an AI assistant designed to understand large, complex projects. It will index the entire codebase and build the knowledge graph (Edge et al., 2024). When users ask questions, Cody's search engine extracts the most relevant code section and context, feeds them into an LLM's context to help them understand the background information, and then generates precise answers. This RAG-driven architecture enables Cody to handle large, complex cross-file questions and avoid the "hallucination" problem common to general-purpose LLMs (Sourcegraph, 2023).

- **GitHub Copilot Workspace:** Evolving from GitHub Copilot, Workspace aims to create an end-to-end "idea to code" AI-native environment (GitHub, 2023). Its capabilities go far beyond auto-completion in a single file or simple chatbot interaction. It can understand the structure of the entire repository and then convert human instructions into complete action plans. This includes creating files, modifying code, and even writing tests. Of course, Copilot can also write project documentation and answer questions about the user's code.

4.3. Revolutionary Impact

These tools directly analyze and understand source code, providing developers with an effective way to understand and maintain code with documentation. They can instantly answer specific questions and automatically complete unimportant details, significantly reducing the time-consuming nature of documentation maintenance.

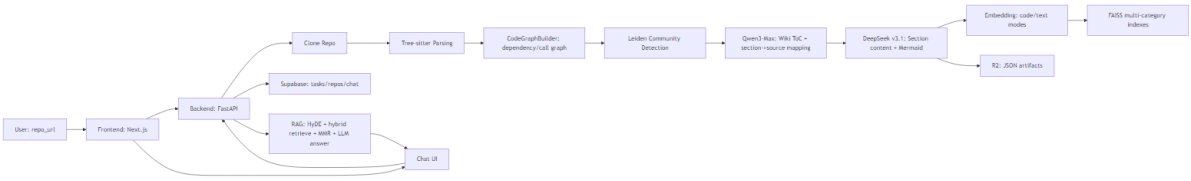
5. Proposed design and system

5.1. System Overview

GitReader is an intelligent web application designed to bridge the gap between complex source code and documentation by generating structured wiki content and leveraging an intelligent agent. The system follows a modular three-tier architecture:

1. **Frontend Layer**: A responsive web interface for repository submission, documentation browsing, and interactive Q&A
2. **Backend Layer**: A Python-based FastAPI service that orchestrates repository ingestion, AST parsing, community detection, and the AI generation pipeline
3. **AI & Persistence Layer**: A hybrid engine integrating multiple Large Language Models (LLMs) via OpenRouter, vector search indices (FAISS), and multi-cloud storage (Supabase, Cloudflare R2)

This design couples static structural documentation with dynamic, evidence-based dialogue to ensure an accurate, context-explicit understanding of the repository.



5.2. System Components

This system contains four primary components:

1. **Frontend (Next.js + Tailwind CSS + Vercel)**
   The frontend is deployed on **Vercel**, providing a React-based UI to visualize the **wiki content** and the agent's reasoning steps in real time, and **Evidence Cards** for RAG traceability.
2. **Backend (FastAPI + Docker + Fly.io)**
   Deployed on **Fly.io**, the backend manages **high-concurrency** tasks and **task-level isolation**. It handles repository cloning **and AST parsing** while serving as the execution environment for the Agent's tool-calling logic.

3. **AI Agent & Model Orchestration Layer**
   To optimize both reasoning depth and cost-efficiency, the system employs a different model for different tasks:

- **Gemini-2.5-Flash (Agent Brain):** Acts as the primary controller for task planning, HyDE query expansion, and agentic tool selection (deciding when to invoke RAG vs. Lexical search).
- **Qwen-3-235B:** Specialized in generating high-level hierarchical Tables of Contents (ToC) (Bai et al., 2023).

4. **Storage & Tool Layer (RAG & Persistence)**
   This layer provides the "external memory" and tools for the Agent:

- **RAG Tool:** A modular retrieval engine combining **FAISS (0.6)** and **BM25 (0.4)** with **MMR (λ=0.5) re-ranking** to provide diverse evidence to the Agent.
- **Supabase:** Manages relational data, task states, and chat metadata.
- **Cloudflare R2**: A scalable object store for heavy artifacts like Wiki JSON structures and chat snapshots**.**
- **Persistent Volumes:** Local FAISS indices and repositories' source code are stored on Fly.io volumes to ensure data survives when the container restarts

5.3. User Interaction Flow

The workflow is divided into an automated processing phase and an interactive understanding phase.

1. **Repo handling:** Upon receiving a URL, the system clones the repo, builds a Code Graph via Tree-sitter, and applies Leiden community detection to group functionally coherent modules.
2. **Wiki Generation:** The system generates a structured Wiki based on a `valid_file_list` to eliminate path hallucinations, storing the result as JSON in R2 for rapid frontend loading.
3. **Agentic Q&A:** Instead of simple retrieval, users interact with an AI Agent. The Agent analyzes the query, plans a multi-step solution (visible via `LiveStepFlow`), invokes relevant tools (RAG or Grep), and synthesizes an evidence-based answer

5.4. Pipeline Refinement

GitReader's core advantage lies in its structure-driven generation scheme and agent-centric interaction logic. This section will focus on the implementation methods of each key stage.

5.4.1. Wiki Generation Pipeline (Structure-First Approach)

To eliminate path illusion and ensure the systematic nature of the documents, the system adopts a two-stage process of "planning first, then generating"：

1. **Wiki Structure Generation**
    - **RepoMap Construction:** Construct a global context that includes a directory tree and high-weight file identifiers.
    - **Code Graph & Community Discovery:** Using Tree-sitter to parse the code and construct a dependency graph. The Leiden algorithm is applied to explore the functional communities.
    - **Summarization:** Generate a functional summary for each community as a macro-level routing signal.
    - **Constrained Planning:** The driver Gemini-2.5-Flash combines community information with the valid_file_list to generate a JSON-formatted directory tree, ensuring that all referenced files actually exist.

2. Wiki Page Content Generation

- **Context Assembly:** Collect associated files for each chapter and extract dependencies and call relationships from the Code Graph.
- **Parallel Generation:** The **ThreadPoolExecutor** is used to concurrently call -Qwen3-235B to synthesize Markdown pages and Mermaid charts based on metadata and context.

5.4.2. Knowledge Indexing (Embedding & Persistence)

The system performs semantic markup on code and documentation, providing the Agent with reliable external retrieval tools:

- **Document Splitting:**
  1. **Code files:** Semantic slicing (such as function and class domains) is performed based on **Tree-sitter AST** to ensure the syntactic integrity of code blocks**.**
  2. **Text files:** Use RecursiveCharacterTextSplitter（chunk_size=2000）to process documents such as README。
- **Embedding Model:** Use the **text-embedding-3-small** model to batch process（batch_size=300）。
- **FAISS Persistence:** Build a FAISS vector index organized by category (code/text) and persist it to **Fly.io Persistent Volumes** to achieve zero-cost recovery after service restart.

5.4.3.Agentic RAG Tool & Reasoning Cycle

During the dialogue phase, RAG is encapsulated as an **atomic tool** (rag_tool.py) of the Agent. The Agent dynamically invokes this tool to obtain evidence based on the user's intent:

- **Input:** User queries, conversation history, FAISS & BM25 indexes.
- **Output: Evidence cards** with precise source code references and combined with the Agent's response.
- **Method (The Tool Logic):**
  1. **Query Expansion (HyDE)**
  2. **Hybrid Retrieval**
  3. **Diversity Re-ranking (MMR)**
- **Agent Synthesis: Gemini-2.5-Flash** acts as the decision-making brain, demonstrating its reasoning process—"think-recall-evaluate evidence-response"—through **LiveStepFlow**. The final answer must be strictly based on the retrieved evidence and accompanied by traceable source citations.

6. Detailed Methodology and Implementation

This section describes in detail how GitReader is implemented in the final system.

6.1. Repository Ingestion and File Filtering

Cloning repository to the persistent volume

GitReader starts from a public GitHub repository URL submitted by the user. The backend clones the repository into a local working directory and records the repository snapshot used for analysis.

Filtering and ignore rules

After cloning, the system filters out irrelevant files and folders such as `node_modules`, `__pycache__`, `venv`, `.git`, hidden files, and binary files. Custom include and exclude rules can also be defined in `repo_config.json`. This step reduces noise, lowers processing cost, and avoids errors caused by unsupported files.

Language identification and parser selection

Files are assigned a language by extension via `EXTENSION_TO_LANGUAGE`. Tree-sitter parsers are loaded dynamically using `tree_sitter_language_pack to` support multi-language repositories (Brunsfeld et al., 2024).

6.2. Repository Structure Extraction and RepoMap Construction

After filtering, GitReader builds a compact repository overview called the **RepoMap**. It contains the **main folder tree**, **important files**, and **key symbols** extracted from the repository. The purpose of RepoMap is to provide a global view of the project without sending the whole repository to the language model.

This structure summary is later used in wiki planning and retrieval. It allows the system to keep **important global context** while controlling token usage, which is especially important for large repositories.

6.3. Code Parsing and Semantic Chunking

AST parsing. TreeSitterParser (`src/ingestion/ts_parser.py`) parses source files into ASTs.

Semantic chunking. Instead of fixed-size splitting, GitReader chunks code by semantic boundaries:

- Prefer top-level function/class nodes
- Extract exact byte ranges (start_byte to end_byte) to preserve syntactic integrity
- Fallback: if no structural nodes are detected, treat the file as a single chunk (this behavior is configurable and can be replaced by a text splitter when needed)

Chunk metadata. Each chunk is stored as a Document with metadata sufficient for traceability:

- source: **repository-relative** file path (recommended for reproducibility)
- start_line, end_line
- node_type
- name (identifier extracted from AST when available)

This metadata is used for source citations in both wiki pages and Q&A answers.

6.4. CodeGraphBuilder (Repository Structural Graph)

Based on the parsing results, GitReader builds a lightweight repository graph. The graph contains **nodes for files, functions, and classes**, and **edges for relationships** such as containment, imports, and simple call links. This graph is not a full compiler-level analysis, but it is sufficient to capture the main repository structure.

- Nodes:
    - File nodes: type="file"
    - Entity nodes: type="class" / type="function"
- Edges:
    - contains: file →entity
    - calls: entity →entity (heuristic)

Call extraction (heuristic)

The system scans AST nodes such as call_expression/call and extracts the callee identifier name. It then links to existing entity nodes via name matching. This approach is intentionally lightweight (not compiler-grade symbol resolution) and trades perfect precision for broad cross-language coverage.

6.5. Community Detection and Module Summarization

To move from file-level to module-level understanding, GitReader applies the **Leiden community-detection** algorithm to the repository graph. (Traag et al., 2019) This group closely relates files and symbols into functional communities. In practice, these communities often match meaningful modules such as UI components, backend services, or utility layers

Community summaries. For each community, representative file/entity names are used to prompt an LLM to produce a concise summary (target: <100 words) describing the functional responsibility of that cluster. The result is a mapping:

- community_id -> summary_text

These summaries are later used as “macro-level routing signals” in the community-first retrieval strategy.

6.6. Embedding and Hybrid Index Construction

To support repository-based Q&A, GitReader builds retrieval indexes from both code and text chunks. In the deployed system, embeddings are generated through a unified AI client layer, and the dense vectors are stored in FAISS. At the same time, the system builds a BM25 index for keyword-based retrieval (Robertson & Zaragoza, 2009).

This hybrid design combines the strengths of semantic search and exact term matching. Dense retrieval is useful for concept-level similarity, while sparse retrieval is better for exact names such as functions, files, and configuration keys.

6.7. Community-first Retrieval and Evidence Selection

For large repositories, I used a **community-first** two-stage retrieval pipeline.

Stage 1 — Macro Search: BM25 over Community Summaries

Given query q, retrieve the top-K communities using BM25 over $\{S_c\}$:

$$C^* = \text{TopK}_c(\text{BM25}(q, S_c))$$

This step narrows retrieval to functionally coherent modules.

Stage 2 — Micro Search: Retrieval within Selected Communities

Restrict candidate chunks to those belonging to C^*, then retrieve evidence primarily via dense vector search (FAISS). Optionally, sparse retrieval and score fusion can be enabled for experiments.

Diversity-aware selection (MMR)

To reduce redundancy in evidence, MMR is applied over candidates:

$$d^* = \arg \max_{d_i \in R \setminus S} \left[\lambda \cdot \text{Sim}(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} \text{Sim}(d_i, d_j) \right]$$

where Simis cosine similarity on embeddings and λcontrols relevance–diversity tradeoff. The selected evidence set is then provided to the generation model, which is instructed to answer strictly based on the evidence and provide source references.

6.8. Backend API and Task Orchestration

The backend is implemented with FastAPI and uses an **asynchronous task-based** design. When a user submits a repository URL, the backend creates a background task for cloning, structure and wiki generation, and index building. The task state is kept up to date in Supabase during execution so the frontend can track progress. The specific backend API structure is in Appendix B.

6.9. Storage and Deployment Integration

GitReader uses different storage services for different types of data. Supabase stores lightweight relational data such as user profiles, task records, repository metadata, and chat history. Cloudflare R2 stores large generated wiki files, including the wiki structure and section content in JSON format. Flyio persistent volume storage is used for cloned repositories and retrieval indexes.

The deployed system uses a Next.js frontend on Vercel and a Dockerized FastAPI backend on Flyio. This design keeps the system scalable and reliable: large files are stored separately, structured metadata remains easy to manage, and vector indexes can survive backend restarts. The specific database structure is presented in Appendix A.

7. Testing and Evaluation

7.1. Testing Environment and Dataset

7.1.1. Deployment Infrastructure

Backend (Fly.io)

- Platform: Fly.io managed containers
- Machine: Fly.io shared-CPU instances with persistent volume mounts (rag_data → /data)
- Runtime: Python 3.10+
- Persistent Storage: Fly Volume mounted at /data; subdirectories `vector_stores/` (FAISS + BM25 indices) and `repos/` (cloned repository snapshots)

Frontend (Vercel)

- Framework: Next.js (React Router SPA via RouterApp.tsx)
- API Base: `NEXT_PUBLIC_API_URL` (backend Fly.io endpoint)
- Authentication: Supabase Auth

External Services

- **Supabase**: PostgreSQL database for tasks, repositories, chat history, user profiles
- **Cloudflare R2**: Object storage for generated Wiki JSON (structure + section pages)
- **OpenRouter**: Unified LLM gateway supporting Qwen, DeepSeek, Gemini models

7.1.2. Test Repositories

Five repositories of varying scale and language mix were selected for evaluation:

Table 7-1 Repositories for Testing

Repository	Primary Language	Approx. Files	Description
reactive-resume	TypeScript/React	~200+	Open-source resume builder (frontend-heavy)
MinerU	Python	~300+	Document parsing and conversion toolkit
autoresearch	Python	~20	LLM-based research automation (small-scale)
zsh-autosuggestions	Shell+Ruby	~30+	Fish-like fast/unobtrusive autosuggestions for zsh
TradingAgents	Python	~50+	Multi-Agents LLM Financial Trading Framework

These repositories are stored under `data/repos/` and referenced in `docker/config/repo_config.json` for file-filtering rules (e.g., excluding lock files, build artifacts, configuration files, etc.).

7.1.3. Model configuration

The system employs multi-model dispatch through OpenRouter:

Table 7- 1 Model configuration for different purposes

Task	Model	Purpose
Wiki Structure	google/gemini-2.5-flash	Generates Table of Contents from code graph
Wiki Content	qwen/qwen3.5-flash-02-23	Section-level Markdown + Mermaid generation
HyDE Generation	google/gemini-2.5-flash	Hypothetical document for retrieval
RAG Answer	qwen/qwen3-235b-a22b-2507	Final answer synthesis

Embeddings	baai/bge-m3	Dense vector embeddings

7.2. Quantitative Performance Analysis

7.2.1. Wiki Generation Efficiency

Table 7-3 Runtime breakdown of the wiki generation pipeline across different repositories

Repo Name	Clone + File Tree	GraphRAG Community Detection	Structure Generation	Content Generation	RAG Index Construction	R2 Upload	Total Runtime
MinerU	50s	262s	100s	550s	190s	22s	17min 54s
TradingAgents	22s	28s	50s	248s	98s	24s	7min 50s
Autoresearch	38s	22s	48s	200s	62s	18s	6min 28s
Zsh-autosuggestions	20s	14s	21s	137s	39s	12s	4min 3s

Table 7-3 reports the execution time for each stage of the wiki generation pipeline across four representative repositories. Overall, content generation accounts for the largest share of the total runtime, while larger repositories such as MinerU require substantially more processing time.

7.2.2. RAG Response Latency

The following values are based on measured averages from benchmark_rag_latency.py across 15 runs: 4 repositories × 3 questions each.

Component	**Measured Latency (Average)**
Vector Index Loading (cache hit)	~0.1s
Vector Index Loading (cold start first load)	~2-5s
HyDE Hypothetical Document Generation	~4-9s (average ~6.5s)
BM25 Sparse Retrieval	<0.05s
FAISS Dense Retrieal	~2.1s(code) + 2.4s (docs) = ~4.5s
Hybrid Fusion + MMR Reranking	<0.1s

Final LLM Generation	~7–17s (average ~10.5s)
Total (HyDE ON)	**16–29s**
Total (HyDE OFF)	**10-22s**

End-to-End Latency Summary by Repository (Average, Including LLM Generation)

Table 7-4 The HyDE rag latency for different repositories

Repository	HyDE ON (s)	HyDE OFF (s)	HyDE Overhead (s)
MinerU	20.06	10.68	9.38
TradingAgents	28.91	22.02	6.89
autoresearch	20.43	14.55	5.88
zsh-autosuggestions	16.29	12.45	3.84
Overall Average	**21.78**	**15.23**	**6.55**

- The retrieval stage (BM25 + FAISS Dense) takes about 4.5s, accounting for roughly 30% of the total HyDE OFF runtime; the remainder is mainly LLM generation time.
- HyDE adds an average latency of about 6.5s, but can improve semantic recall quality.

8. Conclusion and Critical Review

8.1. Summary of Achievements

This project set out to build an **automated Wiki generation and interactive code understanding system** for GitHub repositories. Against the original objectives, the following deliverables have been completed:

Objective	Deliverable	Status
Web-based frontend for repository exploration	Next.js SPA with Supabase Auth, i18n, dark/light theme	✅
Backend API for task management	FastAPI service with async task queue, progress tracking, cancellation	✅
Automated Wiki generation pipeline	Clone → AST → GraphRAG → Structure/Content → R2 + Supabase	✅
RAG-based Q&A	HyDE + Hybrid retrieval + MMR + Streaming SSE	✅
Agent-based deep code understanding	Multi-turn tool-calling agent (rag_search, code_graph, file_read, repo_map)	✅
Code analysis infrastructure	Tree-sitter AST, NetworkX knowledge graph, Leiden community detection	✅
Multi-model orchestration	OpenRouter gateway with task-specific model assignment	✅
Persistent storage architecture	Fly Volume + R2 + Supabase + vector store caching	✅

The system successfully processes repositories of varying scale (from ~20 files to 300+), generating structured Wiki documentation in 14–25 minutes while concurrently producing content.

8.2. Critical Review

8.2.1. Technical Challenges and Solutions

LLM Hallucination — Path Fabrication

A common issue during Wiki content generation was the LLM fabricating file paths that do not exist in the repository — **Hallucination**. This was addressed through `valid_file_list` constraints in the prompt:

- The `WIKI_SECTION_PROMPT` (in `docker/src/prompts.py`) includes the actual file tree and explicit instructions to reference only existing files.
- The `generate_wiki_structure` function in `struct_gen.py` builds the file tree with `generate_file_tree` using filter rules from `repo_config.json`, ensuring the LLM sees only permissible paths.
- Post-generation validation checks that referenced files exist in the repository snapshot.

Concurrent Generation Stability

`WikiContentGenerator.generate()` uses `ThreadPoolExecutor` with configurable `max_concurrency` (default 3, from `repo_config.json`). Challenges included:

- **Filename collision**: Multiple sections could map to the same safe filename; the `_build_filename_map` method with normalisation and suffix numbering resolves this.
- **Partial failure isolation**: One section's LLM timeout must not block or corrupt other sections. The `as_completed` pattern with per-future exception handling ensures this.
- **Progress tracking**: Thread-safe progress accumulation via callback provides accurate Supabase status updates.

8.2.2. Engineering Considerations

Task-Level Directory Isolation

Each generation task operates in its own working directory (`TASK_WORK_ROOT/<task_id>/`), containing:

- `wiki_structure.json` — generated table of contents
- `wiki_section_json/` — per-section content files

This design prevents the race conditions when there are multiple tasks running concurrently. The `cleanup_local_files` function removes the entire task directory after a successful R2 upload, keeping the disk space clean for other tasks while preserving persistent repository clones in `REPO_STORE_PATH`, which contains the repositories' source code and indexing results.

Multi-System Collaboration

I use **Supabase** as the PostgreSQL database, **Cloudflare R2** for wiki structure and content storage, **Fly.io** to host the core backend Docker, and **Vercel** for the frontend. This decoupled architecture eliminates the need for manual server maintenance and effectively prevents the common single-server bottlenecks.

8.3. Learning Experiences

8.3.1. AI Agent Implementation

Through this project, I learned the core principles of agent implementation and operation, including multiple-round inference, task planning, and tool integration. I gained a deeper understanding of how agents autonomously analyze user requests, determine appropriate responses link and analysis logic, and how to leverage existing tools to obtain accurate context and relevant information. This project also enhanced my understanding of how agents can improve response accuracy through structured planning and contextual information retrieval.。

8.3.2. Vector Store Caching

Through building the retrieval pipeline, I learned that embedding strategy impact the answer quality a lot. At the beginning, I embedded code files and text files together. However, I found that text content always dominated the retrival results, which made the system less effective at identifying complete code logic. Then, I changed the pipeline design to process code and text separately. I also applied semantic chunking to code files, so that functions and classes could be kept into a complete units.

8.3.3. GraphRAG Integration

By improving the wiki generation and repository understanding process, I found that file-level information alone was insufficient. In the early stages, the system primarily relied on file content and directory structure, but the lack of information about relationships between files resulted in unsatisfactory performance. This made it difficult to capture how different parts of the repository collaborated. To address this issue, I chose graphrag, which models the dependencies and connections between files and code entities, allowing LLM to obtain more detailed information when generating the wiki's structure and content.

8.4. Future Work

- **Enhancing the Agent:** Future iterations will focus on equipping the agent with a broader set of tools and "deep thinking" capabilities. This will allow for a more profound understanding of user questions and the project's source code, leveraging specialized tools to improve both the logical coherence and factual grounding of its responses
- **Optimizing Wiki Generation:** I plan to continue optimizing the wiki generation logic to better handle projects of different sizes. Currently, the system tends to generate too much content for the small project. Future updates will focus on more accurately extracting key project information.
- **Improving UI/UX:** Refining the page layout and structural design of the web interface to enhance user experience.

9. Appendices

9.1. Appendices A Monthly Logs

- **September 2025**: Submit project plan. Conduct feasibility analysis and draw preliminary architecture design sketches.

- **October 2025**: Complete preliminary literature review. Begin defining system architecture prototypes.

- **November 2025**: Complete backend API design and implement repository cloning, vector index building, and wiki generation functions.

- **December 2025**: Frontend implementation completed and successfully deployed on the Vercel platform. API integration testing completed, and some optimization issues identified.

- **January 2026**: Backend interface testing completed; Supabase and Cloudflare R2 configuration completed, and tests passed.

- **February 2026**: RAG configuration completed, packaged into a tool for the agent to use; deploy the backend Docker on fly.io, tests passed.

- **March 2026**: Conduct code testing and result analyses; invite the real users to experience the website and make some updates based on their feedback; complete the final report and prepare demo, poster, and PowerPoint.

9.2. Appendices B Database Schema and Cloud Storage

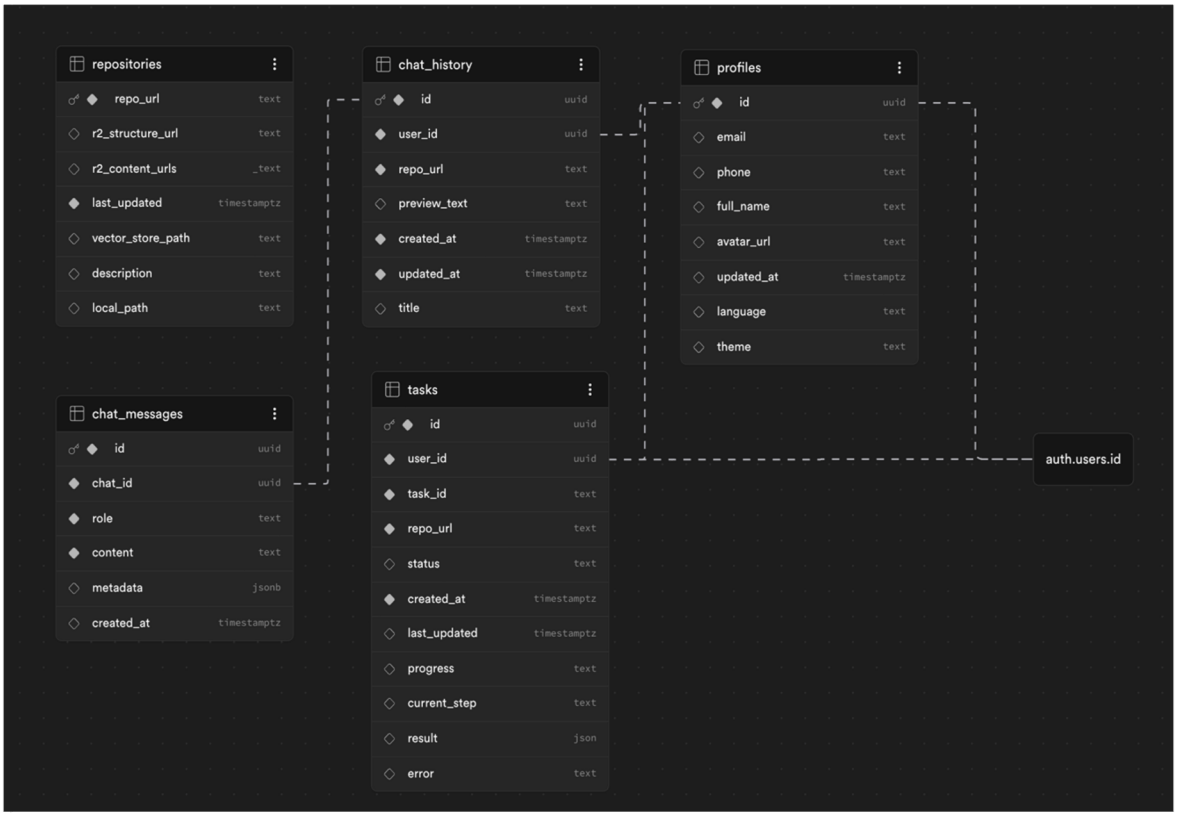


Figure 1 Supabase Database schema diagram

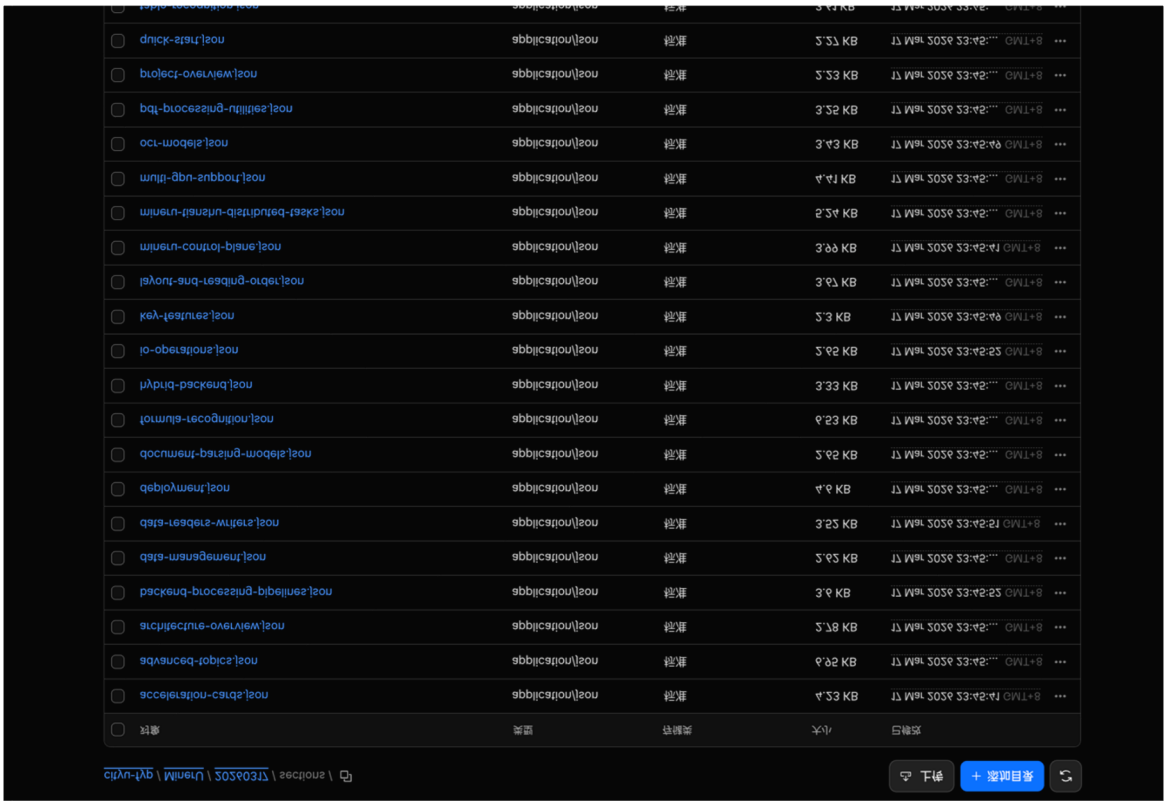


Figure 2 Cloudflare R2 Structure

9.3. Appendices C Backend API List

Table 9-1 Backend API List

Endpoint	Purpose
`POST /generate`	Create an async wiki-generation task for a repo; on cache hit, mark as completed with existing docs
`POST /task/{task_id}`	Fetch status and progress for a task
`POST /tasks`	List all tasks for a given `user_id`
`POST /task/{task_id}/cancel`	Cancel a task that is still processing
`DELETE /task/{task_id}`	Delete a task record
`GET /health`	Health check
`GET /profile`	Get user profile
`PATCH /profile`	Update user profile
`POST /file/content`	Read file contents from a synced repo by `repo_url` and relative `file_path`
`POST /chat`	Repository-grounded RAG Q&A with persistent chat sessions
`POST /chat/stream`	Same as `/chat`, streamed via SSE (retrieval and generation events)
`GET /chat/repos`	List repositories available for chat
`GET /chat/history`	List chat sessions for a `user_id`
`GET /chat/messages/{chat_id}`	Load all messages in a chat session
`DELETE /chat/history/{chat_id}`	Delete a chat session and its messages
`POST /agent/chat`	Multi-step agent Q&A (planning, tools, provenance, optional Mermaid)
`POST /agent/chat/stream`	Agent Q&A over SSE (planning, tool calls, evaluation, synthesis events)

10. References

Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., ... & Zhu, T. (2023). Qwen technical report. *arXiv preprint arXiv:2309.16609.*

Brunsfeld, M., et al. (2024). Tree-sitter: An incremental parsing system for programming tools. *GitHub*. https://github.com/tree-sitter/tree-sitter

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P. E., ... & Jégou, H. (2024). The Faiss library. *arXiv preprint arXiv:2401.08281.*

Edge, D., Trinh, H., Cheng, N., Bradley, J., & Chao, A. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130.*

Gao, L., Ma, X., Lin, J., & Callan, J. (2023). Precise zero-shot dense retrieval without relevance labels. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 535–549.

GitHub. (2023). GitHub Copilot Workspace: The AI-native developer environment. *GitHub Next*. https://githubnext.com/projects/copilot-workspace

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

O'Dell, J. (2023). What is Docs as Code? *The ReadMe Project, GitHub*. https://github.com/readme/guides/docs-as-code

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

Spotify Engineering. (2020). Backstage: Taming the complexity of our microservices and infrastructure. *Spotify Engineering Blog*. https://engineering.atspotify.com/2020/03/taming-the-complexity-of-our-microservices-and-infrastructure

Swimm. (n.d.). Continuous Documentation: Keep your docs in sync with code. https://swimm.io

Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1), 5233.

Wu, Y., Wan, Y., Chu, Z., Zhao, W., Liu, Y., Zhang, H., ... & Yu, P. S. (2024). Can large language models serve as evaluators for code summarization? *IEEE/ACM International Conference on Software Engineering*.